



南開大學
Nankai University

计算机学院
数据安全课程实验报告

半同态加密应用实践

姓名：孔祥昊

2311439

专业：计算机科学与技术

2026 年 4 月 10 日

目录

1 实验要求	2
2 实验原理	2
2.1 同态加密 (Homomorphic Encryption, HE)	2
2.2 半同态加密 (Partially Homomorphic Encryption, PHE)	2
2.3 Paillier 算法	2
2.3.1 密钥生成	2
2.3.2 加密过程	3
2.3.3 解密过程	3
2.3.4 关键同态性质	3
3 实验	3
3.1 基于 phe 库的加法和标量乘法验证	3
3.2 隐私信息获取实验	5
3.3 拓展实验	6
4 Github 链接	8

1 实验要求

- 基于 Paillier 算法实现隐私信息获取：从服务器给定的 m 个消息中获取其中一个，不得向服务器泄露获取了哪一个消息，同时客户端能完成获取消息的解密。
- **拓展实验：**在客户端保存对称密钥 k ，在服务器端存储 m 个用对称密钥 k 加密的密文，通过隐私信息获取方法得到制定密文后能解密得到对应的明文。

2 实验原理

2.1 同态加密 (Homomorphic Encryption, HE)

同态加密是一种特殊的公钥加密方案。它允许在不解密的情况下，直接对密文进行特定的代数运算，其运算结果解密后与直接对明文执行相应运算的结果保持一致。

从代数角度定义：设 $\mathcal{M}$ 为明文空间， $\mathcal{C}$ 为密文空间。若加密算法为 E ，解密算法为 D ，对于明文空间上的某种运算 $\odot$ ，如果存在密文空间上的对应运算 $\otimes$ ，使得对于任意 $m_1, m_2 \in \mathcal{M}$ ，均满足：

$$D(E(m_1) \otimes E(m_2)) = m_1 \odot m_2 \quad (1)$$

则称该加密方案对运算 $\odot$ 具有同态性。

2.2 半同态加密 (Partially Homomorphic Encryption, PHE)

半同态加密是指仅支持在密文空间上执行一种特定运算（如加法或乘法）且支持无数次该运算的同态加密方案。根据其支持的算子不同，通常分为两类：

- **加法同态加密：**支持密文的同态加法。满足 $D(E(m_1) \otimes E(m_2)) = m_1 + m_2$ 。典型的算法包括 Paillier 和 ElGamal 的变体。
- **乘法同态加密：**支持密文的同态乘法。满足 $D(E(m_1) \otimes E(m_2)) = m_1 \cdot m_2$ 。典型的算法如未经填充的 RSA 算法。

2.3 Paillier 算法

Paillier 算法是一种基于复合剩余类合数剩余问题 (Decisional Composite Residuosity Assumption, DCRA) 的加法半同态概率加密方案。其具体流程如下：

2.3.1 密钥生成

1. 选取两个等长的大素数 p 和 q ，计算 $n = pq$ 以及 $\lambda = \text{lcm}(p-1, q-1)$ 。
2. 定义函数 $L(x) = \frac{x-1}{n}$ 。
3. 选取随机整数 $g \in \mathbb{Z}_{n^2}^*$ ，通常取 $g = n+1$ 。
4. 计算模反元素 $\mu = (L(g^\lambda \pmod{n^2}))^{-1} \pmod{n}$ 。
5. 公钥为 (n, g) ，私钥为 (λ, μ) 。

2.3.2 加密过程

对于明文 $m \in \mathbb{Z}_n$ ，选取随机数 $r \in \mathbb{Z}_n^*$ ，计算密文 c ：

$$c = g^m \cdot r^n \pmod{n^2} \quad (2)$$

2.3.3 解密过程

对于密文 $c \in \mathbb{Z}_{n^2}^*$ ，通过私钥还原明文 m ：

$$m = L(c^\lambda \pmod{n^2}) \cdot \mu \pmod{n} \quad (3)$$

2.3.4 关键同态性质

Paillier 算法在隐私信息获取 (PIR) 中利用了以下核心性质：

- 密文同态加法：

$$E(m_1) \cdot E(m_2) \equiv E(m_1 + m_2) \pmod{n^2} \quad (4)$$

- 密文同态标量乘法：

$$E(m)^k \equiv E(k \cdot m) \pmod{n^2} \quad (5)$$

3 实验

3.1 基于 phe 库的加法和标量乘法验证

基于以下提供的代码，我们进行验证。

```

1 from phe import paillier # 开源库
2 import time # 做性能测试
3
4 ##### 设置参数
5 print("默认私钥大小：", paillier.DEFAULT_KEYSIZE)
6 #生成公私钥
7 public_key, private_key = paillier.generate_paillier_keypair()
8 # 测试需要加密的数据
9 message_list = [3.1415926,100,-4.6e-12]
10
11 ##### 加密操作
12 time_start_enc = time.time()
13 encrypted_message_list = [public_key.encrypt(m) for m in message_list]
14 time_end_enc = time.time()
15 print("加密耗时s：",time_end_enc-time_start_enc)
16 print("加密数据 (3.1415926) :",encrypted_message_list[0].ciphertext())
17
18 ##### 解密操作
19 time_start_dec = time.time()
20 decrypted_message_list = [private_key.decrypt(c) for c in encrypted_message_list]
21 time_end_dec = time.time()

```

```

22 print("解密耗时s: ",time_end_dec-time_start_dec)
23 print("原始数据 (3.1415926) :",decrypted_message_list[0])
24
25 ##### 测试加法和乘法同态
26 a,b,c = encrypted_message_list # a,b,c分别为对应密文
27 a_sum = a + 5 # 密文加明文, 已经重载了+运算符
28 a_sub = a - 3 # 密文加明文的相反数, 已经重载了-运算符
29 b_mul = b * 6 # 密文乘明文,数乘
30 c_div = c / -10.0 # 密文乘明文的倒数
31
32 print("a+5 密文:",a.ciphertext()) # 密文纯文本形式
33 print("a+5=",private_key.decrypt(a_sum))
34 print("a-3",private_key.decrypt(a_sub))
35 print("b*6=",private_key.decrypt(b_mul))
36 print("c/-10.0=",private_key.decrypt(c_div))
37
38 ##密文加密文
39 print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a+b))
40 #报错, 不支持a*b, 即两个密文直接相乘
41 #print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a*b))

```

得到如下结果:

```

root@f9c97913a89b:/workspaces/lab# python lab1/source/test1.py
默认私钥大小: 3072
加密耗时s: 0.5517652034759521
加密数据 (3.1415926) : 100136169073487185167702909406931158508976377570354138312555868634907391713493769106082470449948365612815449306179371465292913120474000876
28050026701287969332363765485718936110981245346390097615718829218528121218328619324121580412043587101874576267902491695259870999110510494137265987129009796306285
6930714884446736834195306540088649461323163279395496376047372976517392547744828049019295203063963761938837278365192266924475298652211417340471319042314401515544
86712726776700580631486856244498117521170604382118975983381055719250286907470232306840307135156248476275697063487315792974371196010004377724874699294946111134896
632105143313846514076271546739019889504844487530901694715283567382994015729274035794175038167517713130900252415140722565227275983246721014836987045006617313216
6583464574195580002930401625089943451500480711178726277655772128973909856087495764106603594398260656147161212215067387541884780466976141804395276759598324590464
6574826788408148863139792163038507818546234917743989001991308387314048143485940259849991153044776286770929007086111049117414201282299935787160060871974731916666
31960323605222972164235648595467562827932736477867879769122368614499406938656027680085665440991069060617460439021348526440452015710340398506241511339330946830421
59007409501730323826442937997608065614408182704446399650592540105182610293919148694305616151147214658645247877555706113875419045756293143993042928243680397859779
3990868995581867983082835049151684763478126590269740730402313584238557132592271376545741648114353787410321145854429428063864067620014905928280097518417898665222
89721604796233375101707679408657556573854541492661051488392540717147713281076638023710269782659592616759220587714962504903660933718048132585567935952906576198215
245339844342987908992631805509883563719956184018967699157337880153471632666161823236297995386369612940
解密耗时s: 0.15034842491149982
原始数据 (3.1415926) : 3.1415926
a+5 密文: 1001361690734871851677029094069311585089763775703541383125558686349073917134937691060824704499483656128154493061793714652929131204740008762805002670128
76933236376548571893611098124534639009761571882921852812121832861932412158041204358710187457626790249169525987099911051049413726598712900979630628569307148844467
7368341953065400886494613231632793954963760473729765173925477448280490192952030639637619388372783651922669244752986522114173404713190423144015155448671272677670
05806314868562444981175211706043821189759833810557192502869074702323068403071351562484762756970634873157929743711960100043777248746992949461111348966321051433138
4651407627154673901988950484448753090169471528356738299401572927403579417503816751771313090025241514072256522727598324672101483698704540066173132166583464574195
5800029304016250899434515004807111787262776557721289739098560874957641066035943982606561471612122150673875418847804669761418043952767595983245904646574826788408
1488631397921630385078185462349177439890019913083873140481434859402598499911530447762867709290070861110491174142012822999357871600608719747319166663196032360522
29721642356485954675628279327364778678797691223686144994069386560276800856654409910690606174604390213485264404520157103403985062415113393309468304215900740950173
0323826442937997608065614408182704446399650592540105182610293919148694305616151147214658645247877555706113875419045756293143993042928243680397859779399086899558
18679830828350491516847634781265902697407304023135842385571325922713765457416481143537874103211458544294280638640676200149059282800975184178986652228972160479623
33751017076794086575565738545414926610514883925407171477132810766380237102697826595926167592205877149625049036609337180481325855679359529065761982152453398443429
87908992631805509883563719956184018967699157337880153471632666161823236297995386369612940
a+5= 8.1415926
a-3 0.14159260000000007
b*6= 600
c/-10.0= 4.6e-13
True

```

图 3.1: 加法和标量乘法验证结果

如图3.1所示, 加密解密操作正常, 输出为 True, 结果验证无误。

随后去掉代码最后一行注释, 验证 Paillier 算法是否支持乘法运算:

```

Traceback (most recent call last):
  File "/workspaces/lab/lab1/source/test1.py", line 41, in <module>
    print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a*b))
  File "/usr/local/lib/python3.10/site-packages/phe/paillier.py", line 508, in __mul__
    raise NotImplementedError('Good luck with that...')
NotImplementedError: Good luck with that...

```

图 3.2: 乘法报错结果

如图3.2所示, Paillier 算法不支持同态乘法, 验证完毕。

3.2 隐私信息获取实验

本实验利用 Paillier 加法同态加密方案实现计算型隐私信息获取。其核心逻辑在于通过加密的选择向量对服务器数据进行线性组合，使得服务器在无法获知查询索引的前提下计算出结果。具体流程如下：

具体实现方案

1. 初始化与密钥生成 (Client)

用户生成公私钥对 (pk, sk) 。将公钥 pk 发送至服务器，用于密文空间下的同态运算；私钥 sk 由用户本地保留，用于结果解密。

2. 构造查询向量 (Client)

设服务器拥有 m 个消息，用户欲获取第 i 个消息。用户在本地构造一个单位向量 $V = [v_1, v_2, \dots, v_m]$ ，其定义如下：

$$v_j = \begin{cases} 1, & j = i \\ 0, & j \neq i \end{cases} \quad (6)$$

用户对向量每一位进行加密，得到密文向量 $E(V) = [E(v_1), E(v_2), \dots, E(v_m)]$ 并发送给服务器。

3. 同态掩码计算 (Server)

服务器拥有消息列表 $M = [M_1, M_2, \dots, M_m]$ 。收到 $E(V)$ 后，服务器执行以下同态运算：

- **同态标量乘法**：利用性质 $E(m)^k = E(m \cdot k)$ ，计算每一项消息与掩码密文的乘积：

$$C_j = E(v_j)^{M_j} \pmod{n^2} \quad (7)$$

- **同态加法聚合**：利用性质 $E(m_1) \cdot E(m_2) = E(m_1 + m_2)$ ，将所有中间结果聚合：

$$C_{result} = \prod_{j=1}^m C_j \pmod{n^2} \quad (8)$$

4. 结果解密 (Client)

服务器返回单一密文 C_{result} 。用户使用私钥 sk 进行解密，恢复出原始消息 M_i 。

基于如下代码，我们进行隐私信息获取的验证及尝试。

```

1 from phe import paillier # 开源库
2 import random # 选择随机数
3
4 ##### 设置参数
5 # 服务器端保存的数值
6 message_list = [100,200,300,400,500,600,700,800,900,1000]
7 length = len(message_list)
8 # 客户端生成公私钥
9 public_key, private_key = paillier.generate_paillier_keypair()
10 # 客户端随机选择一个要读的位置
11 pos = random.randint(0,length-1)
12 print("要读起的数值位置为：",pos)
13

```

```

14 ##### 客户端生成密文选择向量
15 select_list=[]
16 enc_list=[]
17 for i in range(length):
18     select_list.append( i == pos )
19     enc_list.append( public_key.encrypt(select_list[i]) )
20
21 # for element in select_list:
22 #     print(element)
23 # for element in enc_list:
24 #     print(private_key.decrypt(element))
25
26 ##### 服务器端进行运算
27 c=0
28 for i in range(length):
29     c = c + message_list[i] * enc_list[i]
30 print("产生密文：",c.ciphertext())
31
32 ##### 客户端进行解密
33 m=private_key.decrypt(c)
34 print("得到数值：",m)

```

运行结果如下：

```

root@f9c97913a89b:/workspaces/lab# python lab1/source/test.py
要读取的数值位置为： 4
产生密文： 13206689106604069045780098400141969295997449897331416100863398409857992203671342288185437402947914859961900
2035183247893843481166711736333379239780040350970306327895606335551061671687272210018206410584824792238562433640365393
8993540201661462097364962888752825370658756973359513538136023049786638167963112902737475037533817685226014957774165235
8332958701188018235771257552957906701130606703235308334554055974715819334296429683393522167483871409990620468372829688
688124416444612572354706744823025023806791619298554755697613536392047246965377453135124883617075218992570690807314655
9606908995579257858663987582238693954891982190787770889908351299708862804324573694028966142680081874215937321980128682
7517455478063542966025032148027384355911248436503445109554617690932980102899912124416607020871657742621503507083756509
2764464050445457577069356189270032034639641063585556371783826844916134760253841311781079688996812321822471721868545341
3239769753492140871196970164945430912046543240036080918201877492236082167234030869212658489077484373683189152462014727
6944061477269004105931882902945182321641759220612761445666601214441070387749846213490469760525333861777620877322447606
284621460549260895340660906565212209755667458656576507610979163461043111172904193032753767949930694832541250907396732
7493637176222750464426930416769798237550993902746855173422960200185602922487496757814658933778594052812640565511644119
4729847613588067473922190813611487672378392018118228245624756103028154111969047172543057224038617318434052089352667339
354460108705486450608654157206859032395660521887143382463841793584479145618746356477788356128045565291546109838494318
0132275652143294922467262259063075091552102510897736666465067980918045233667418031806059951771638889946365139656664285
063788499832490985727753759314805024352200812655475846748243536030736840679672761285907617
得到数值： 500

```

图 3.3: 隐私信息获取结果

如图3.3所示，我们成功获取了隐私信息，验证正确。

3.3 拓展实验

本次实验采用 AES 对称加密方式来实现。代码如下：

```

1 from phe import paillier
2 from cryptography.fernet import Fernet
3 import random
4
5 ##### 1. 初始化设置与对称加密 (Client 端操作模拟)
6 print("=== 初始化阶段 ===")
7 original_messages = ["Secret_100", "Secret_200", "Secret_300", "Secret_400",
    "Secret_500",

```

```

8         "Secret_600", "Secret_700", "Secret_800", "Secret_900",
          "Secret_1000"]
9 length = len(original_messages)
10
11 # 客户端生成对称密钥 k (AES)
12 symmetric_key = Fernet.generate_key()
13 cipher_suite = Fernet(symmetric_key)
14 print("客户端生成的对称密钥 k (保存于本地):", symmetric_key)
15
16 # 模拟: 用对称密钥 k 加密所有明文, 并转化为大整数存入服务器
17 server_stored_integers = []
18 for msg in original_messages:
19     # 1. 对称加密, 得到字节流 (bytes)
20     encrypted_bytes = cipher_suite.encrypt(msg.encode('utf-8'))
21     # 2. 将字节流转换为大整数 (int), 以便进行 Paillier 同态运算
22     encrypted_int = int.from_bytes(encrypted_bytes, byteorder='big')
23     server_stored_integers.append(encrypted_int)
24
25 print("服务器端存储的数据已全部转换为对称加密后的大整数。")
26
27 ##### 2. PIR 阶段: 客户端生成公私钥及查询向量 (Client 端操作)
28 print("\n=== PIR 查询阶段 ===")
29 # 客户端生成 Paillier 公私钥
30 public_key, private_key = paillier.generate_paillier_keypair()
31
32 # 客户端随机选择一个要读取的位置
33 pos = random.randint(0, length - 1)
34 print(f"客户端想要获取的位置为: {pos} (预期明文: {original_messages[pos]})")
35
36 # 生成密文选择向量
37 select_list = []
38 enc_list = []
39 for i in range(length):
40     select_list.append(i == pos) # 只有 pos 位置是 1(True), 其余是 0(False)
41     enc_list.append(public_key.encrypt(select_list[i]))
42
43 ##### 3. PIR 阶段: 服务器端进行同态掩码运算 (Server 端操作)
44 print("\n=== 服务器运算阶段 ===")
45 c = 0
46 for i in range(length):
47     # server_stored_integers[i] 是对称加密后的大整数, enc_list[i] 是同态加密的 0 或 1
48     # 标量乘法: 整数 * 密文
49     c = c + server_stored_integers[i] * enc_list[i]
50
51 print("服务器运算完成, 产生聚合密文 (返回给客户端)。")
52
53 ##### 4. 解析阶段: 客户端进行解密 (Client 端操作)
54 print("\n=== 客户端解密阶段 ===")
55 # 第一步: 用 Paillier 私钥解密, 得到目标信息的对称密文 (大整数形式)

```



```

56 m_int = private_key.decrypt(c)
57
58 # 第二步：将大整数转回字节流
59 # 计算所需的字节长度：(位长度 + 7) // 8
60 byte_length = (m_int.bit_length() + 7) // 8
61 m_bytes = m_int.to_bytes(byte_length, byteorder='big')
62
63 # 第三步：使用本地保存的对称密钥 k 进行 AES 解密
64 final_plaintext = cipher_suite.decrypt(m_bytes).decode('utf-8')
65
66 print("1. Paillier解密得到的大整数:", m_int)
67 print("2. 转换回的对称密文字节流:", m_bytes[:30], "... (截断显示)")
68 print("3. 最终通过对称密钥解密得到的明文:", final_plaintext)

```

1. **引入 Fernet**: Fernet 是 Python 中提供的高级对称加密工具，底层封装了 AES-128-CBC 和 HMAC，保证了数据的机密性和完整性。
2. **数据类型桥接 (int.from_bytes / to_bytes)**: 因为 Paillier 是基于大数模幂运算的算法，它无法识别计算机存储的二进制字节 (Bytes)。我们必须将对称密文字节流整体映射为一个极大的整数（通常几百到上千位）进行计算，解密后再无损还原，这就保证了密码学层面的无缝衔接。
3. **双重安全保障**: 此时服务器即使攻破了 Paillier 加密，它拿到也只是被 AES 加密过的乱码；而在 PIR 传输过程中，拦截者既拿不到查询索引，也拿不到明文内容。

运行代码结果如下：

```

root@e735f01fbb70:/workspaces/lab# python lab1/source/advanced.py
=== 初始化阶段 ===
客户端生成的对称密钥 k (保存于本地): b'-cT-t7sEnjYlZDt03d50vF41bbE01yUDLThD_0oxbc='
服务器端存储的数据已全部转换为对称加密后的大整数。

=== PIR 查询阶段 ===
客户端想要获取的位置为: 1 (预期明文: Secret_200)

=== 服务器运算阶段 ===
服务器运算完成，产生聚合密文 (返回给客户端)。

=== 客户端解密阶段 ===
1. Paillier解密得到的大整数: 2689473345859782018765882615531651844841260133022509205937189076590952883827343477284410602243831826564201779917145588509819101997113609162135565371119214783472005178979552054375825942900730953713693738609247219360423568138540568454119243069
2. 转换回的对称密文字节流: b'gAAAAABp2JeXSoSB0VcXyqhb11JSA' ... (截断显示)
3. 最终通过对称密钥解密得到的明文: Secret_200
root@e735f01fbb70:/workspaces/lab# python lab1/source/advanced.py
=== 初始化阶段 ===
客户端生成的对称密钥 k (保存于本地): b'Np2Rp7qms_10TTX29d2ewmqo7MA0L6GpPeBamM4GZ8M='
服务器端存储的数据已全部转换为对称加密后的大整数。

=== PIR 查询阶段 ===
客户端想要获取的位置为: 0 (预期明文: Secret_100)

=== 服务器运算阶段 ===
服务器运算完成，产生聚合密文 (返回给客户端)。

=== 客户端解密阶段 ===
1. Paillier解密得到的大整数: 268947334585978201876588272326839278008911579876944745231933177293591834616880686115670757468736235682802740730181237916162614612779549462409889492864132895812314007352505713301167617557903315421120714125750324269763136946327764007712668989
2. 转换回的对称密文字节流: b'gAAAAABp2JjXoJI5ogXiyWnVBRcJzW' ... (截断显示)
3. 最终通过对称密钥解密得到的明文: Secret_100

```

图 3.4: 拓展实验结果

如图3.4所示，成功实现了基于 AES 对称加密的隐私信息获取。

4 Github 链接

完整代码访问[GitHub 链接](#)获取。